

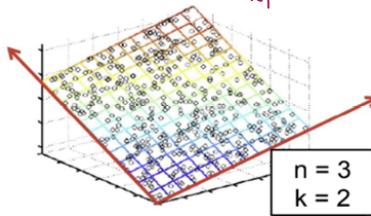
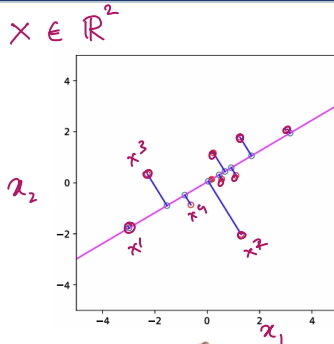
The Dimensionality Reduction Problem

Real Data is High-Dimensional

- MNIST digit image: $28 \times 28 = 784$ dims
- Face image 64×64 : **4096** dims
- Gene expression: **20,000+** genes

But Most Variation is Low-Dimensional

Pixels are correlated. Many images look similar. The **effective degrees of freedom** $\ll D$.



Another example

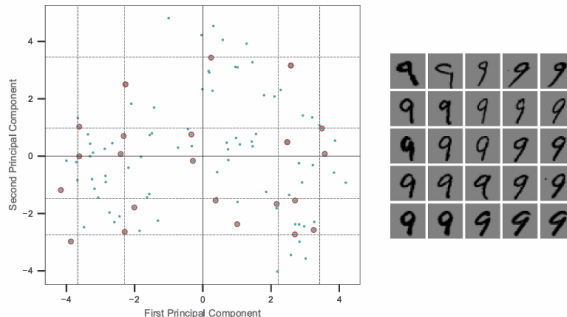


Figure 20.2: An illustration of PCA applied to MNIST digits from class 9. Grid points are at the 5, 25, 50, 75, 95 % quantiles of the data distribution along each dimension. The circled points are the closest projected images to the vertices of the grid. Adapted from Figure 14.23 of [HTF09]. Generated by `pca_digits.ipynb`.

Goal of dimensionality reduction.

1. **Compression** — represent $\mathbf{x} \in \mathbb{R}^D$ as $\mathbf{z} \in \mathbb{R}^L$, $L \ll D$
2. **Visualization** — project to 2D/3D
3. **Preprocessing** — faster downstream ML
4. **Noise reduction** — discard small-variance dims

Methods of dimensionality reductions

1. Linear projection
2. Non-linear projection (Auto-encoder)
3. Manifold learning

Problem setup

Given: Dataset $\{\mathbf{x}_n\}_{n=1}^N$, $\mathbf{x}_n \in \mathbb{R}^D$. Want embedding $\mathbf{z}_n \in \mathbb{R}^L$, $L < D$.

Linear model: Encode and decode with orthogonal matrix $\mathbf{W} \in \mathbb{R}^{D \times L}$:

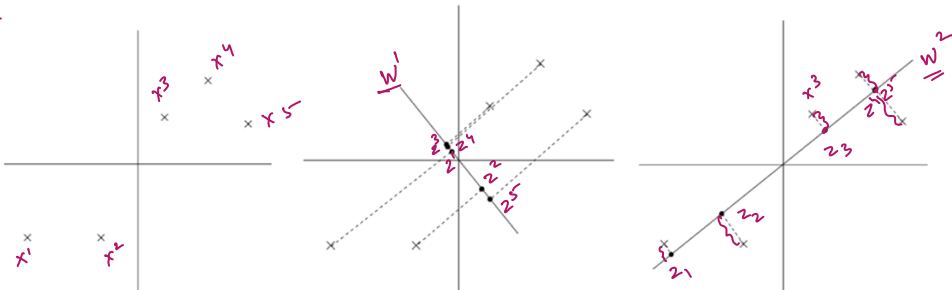
$$\mathbf{z}_n = \mathbf{W}^\top \mathbf{x}_n \quad (\text{encode}), \quad \hat{\mathbf{x}}_n = \underline{\mathbf{W} \mathbf{z}_n} \quad (\text{decode})$$

Characterizing goodness of linear projections

No labelled supervision on the desired z . How do we choose the best z ?

Which projection is better?

$D=2$



A good objective

- ① Minimize reconstruction error (distortion): [sum of the lengths of the blue lines]
 $= \frac{1}{N} \sum_{n=1}^N \|x_n - \hat{x}_n\|^2$

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N} \sum_{n=1}^N \|x_n - \mathbf{W}z_n\|^2 = \frac{1}{N} \|\mathbf{X} - \mathbf{Z}\mathbf{W}^T\|_F^2$$

$$\mathbf{X} = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{bmatrix} \quad x_i \in \mathbb{R}^{D \times 1}$$

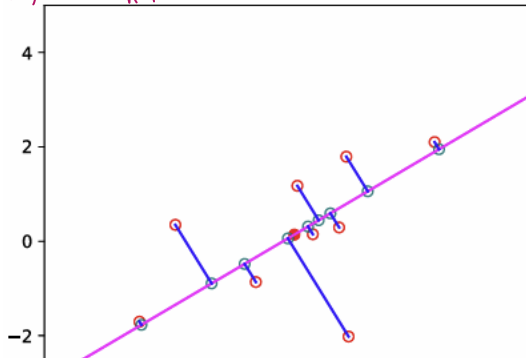
$$\mathbf{Z} = \begin{bmatrix} z_1^T \\ \vdots \\ z_N^T \end{bmatrix}$$

- ② Equivalent to minimizing variance of the projected points.

$$L=1 \quad \sum_{n=1}^N (z_n - \bar{z})^2 = \sum_{n=1}^N (w^T x_n - w^T \bar{x})^2$$

$$\mathbf{W} =_{D \times L} \begin{bmatrix} w_1^T \\ w_2^T \\ \vdots \\ w_L^T \end{bmatrix}$$

$$w_j \in \mathbb{R}^{D \times 1}$$



Optimal Solution

$$\hat{\underline{\mathbf{W}}} = \underset{\underline{\mathbf{W}}}{\operatorname{argmin}} \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \underline{\mathbf{W}} \mathbf{z}_n\|^2$$

Set $\hat{\underline{\mathbf{W}}} = \underline{\mathbf{U}}_L$, where $\underline{\mathbf{U}}_L$ contains the L eigenvectors with the **largest eigenvalues** of the empirical covariance matrix:

$$\underline{\hat{\Sigma}} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c$$

Derivation: 1D Base Case

$$L = 1$$

Since PCA was covered in CS 215. I skipped all the derivations here.
If you have forgotten look at Chapter 20.1.2

Derivation (Summary)

Find the best single direction $\mathbf{w}_1 \in \mathbb{R}^D$, $\|\mathbf{w}_1\| = 1$.

Step 1. Optimal scalar embedding given $\mathbf{w}_1$:

$$\frac{\partial \mathcal{L}}{\partial z_{n1}} = 0 \implies \boxed{z_{n1} = \mathbf{w}_1^\top \mathbf{x}_n} \quad (\text{orthogonal projection})$$

Step 2. Substitute back — minimizing reconstruction error $\equiv$ **maximizing projected variance**:

$$\mathcal{L}(\mathbf{w}_1) = \text{const} - \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 \implies \max_{\|\mathbf{w}_1\|=1} \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1$$

Step 3. Lagrangian optimality:

$$\hat{\Sigma} \mathbf{w}_1 = \lambda_1 \mathbf{w}_1 \implies \mathbf{w}_1 = \text{top eigenvector of } \hat{\Sigma}$$

Induction

Repeat deflating $\hat{\Sigma}$ to get $\mathbf{w}_2 \perp \mathbf{w}_1$, etc. $\Rightarrow \hat{\mathbf{W}} = \mathbf{U}_L$.

High-Dimensional Trick: Gram Matrix

Problem: When $D > N$, computing eigenvectors of $D \times D$ matrix $\mathbf{X}^\top \mathbf{X}$ is expensive.

Solution: Work with the $N \times N$ **Gram matrix** $\mathbf{X}\mathbf{X}^\top$ instead.

If $(\mathbf{X}\mathbf{X}^\top)\mathbf{u} = \lambda\mathbf{u}$, then
 $(\mathbf{X}^\top \mathbf{X})(\mathbf{X}^\top \mathbf{u}) = \lambda(\mathbf{X}^\top \mathbf{u})$.

$$\mathbf{V} = \mathbf{X}^\top \mathbf{U} \Lambda^{-1/2}$$

Normalized eigenvectors of $\mathbf{X}^\top \mathbf{X}$.

When to Use Which

	$D < N$	$D > N$
Use	$\mathbf{X}^\top \mathbf{X}$	$\mathbf{X}\mathbf{X}^\top$
Size	$D \times D$	$N \times N$
Cost	$O(D^3)$	$O(N^3)$

Key Insight

This trick also enables **Kernel PCA** — replace $\mathbf{X}\mathbf{X}^\top$ with any kernel matrix $\mathbf{K}$.

Standardization Matters

Problem: Scale Sensitivity

PCA is sensitive to the **units** of each feature. A feature measured in millimeters vs kilometers will dominate PCA simply due to scale.

Example — Height vs Weight:

- Raw: height range ≈ 25 , weight range ≈ 200
- PC1 essentially just measures weight

Fix: Standardize First

Subtract mean, divide by std (= use **correlation matrix**).

Always standardize when features have different units.

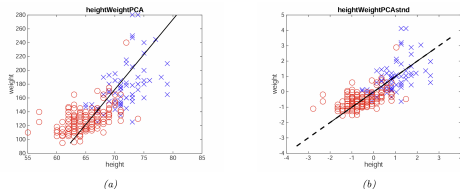


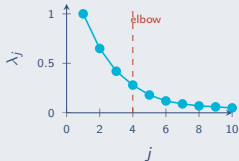
Figure 20.5: Effect of standardization on PCA applied to the height/weight dataset. (Red=female, blue=male.)
Left: PCA of raw data. Right: PCA of standardized data. Generated by [pcaStandardization.ipynb](#).

How Many Components? — Three Strategies

1. Scree Plot

Plot eigenvalues λ_j vs j .

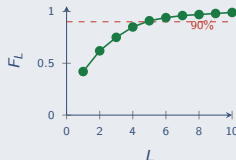
Look for an “**elbow**” where the curve flattens.



2. Variance Explained

$$F_L = \frac{\sum_{j=1}^L \lambda_j}{\sum_{j=1}^{L_{\max}} \lambda_j}$$

Choose L so $F_L \geq 0.90$ (or 95%).



3. Profile Likelihood

Fit a change-point model to eigenvalues. Choose L^* that maximizes:

$$\ell(L) = \sum_k \log \mathcal{N}(\lambda_k | \mu_{1/2}, \sigma^2)$$



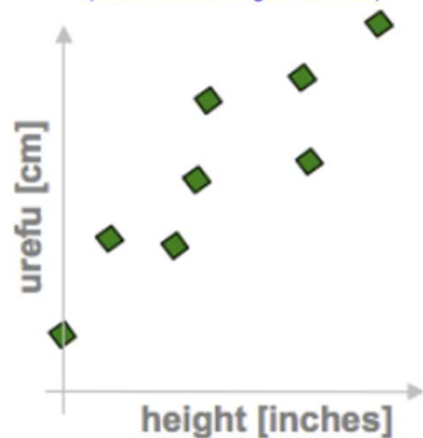
Warning: No U-shaped Test-Set Curve

Unlike supervised learning, PCA reconstruction error on the **test set keeps**

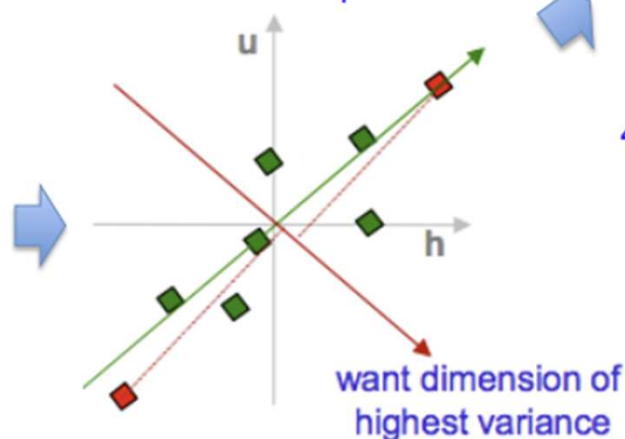
PCA in a nutshell

1. correlated hi-d data

("urefu" means "height" in Swahili)



2. center the points



3. compute covariance matrix

$$\begin{matrix} & h & u \\ h & 2.0 & 0.8 \\ u & 0.8 & 0.6 \end{matrix} \rightarrow \text{cov}(h, u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

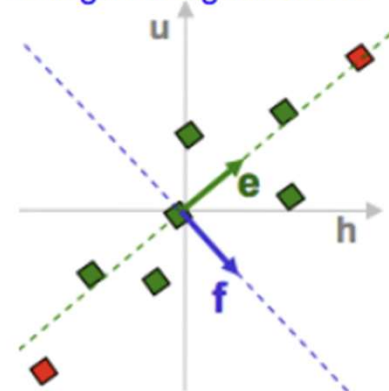
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

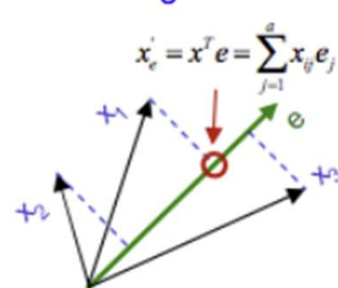
$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

`eig(cov(data))`

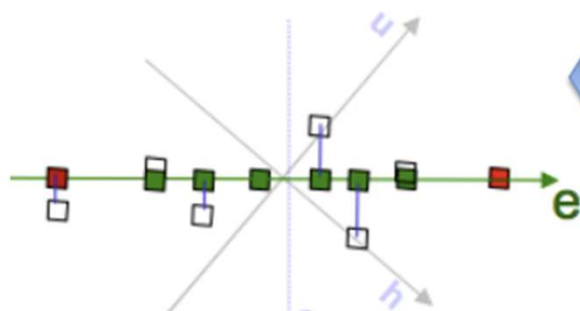
5. pick m < d eigenvectors w. highest eigenvalues



6. project data points to those eigenvectors



7. uncorrelated low-d data



Copyright © 2011 Victor Lavrenko

Eigen faces demo

PCA example: Eigen Faces

input: dataset of N face images



can visualize
eigenvectors:
 m "aspects"
of prototypical
facial features

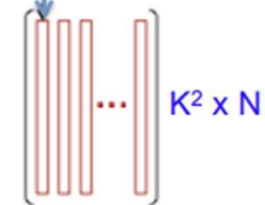


face: $K \times K$ bitmap of pixels



"unfold" each bitmap to
 K^2 -dimensional vector

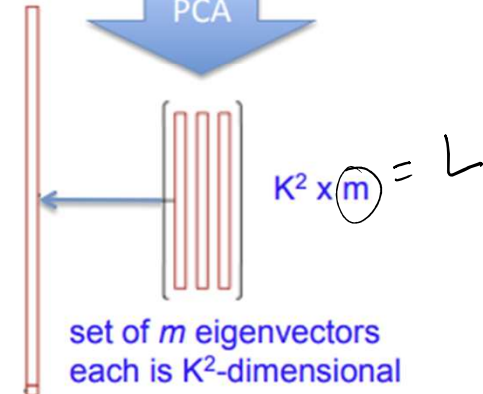
arrange in a matrix
each face = column



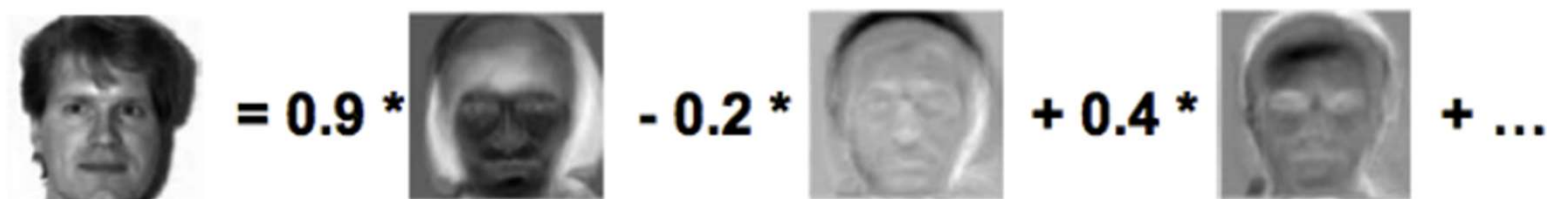
"fold" into a $K \times K$ bitmap



PCA

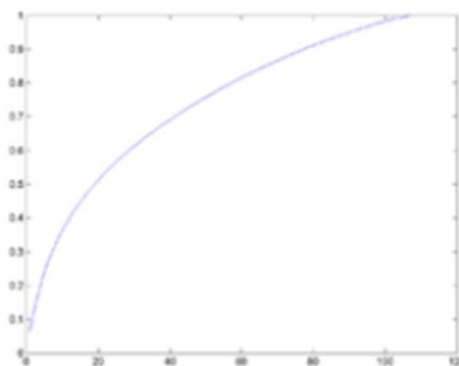


Eigen Faces: Projection


$$\text{Target Face} = 0.9 * \text{Eigenface 1} - 0.2 * \text{Eigenface 2} + 0.4 * \text{Eigenface 3} + \dots$$



- Project new face to space of eigen-faces
- Represent vector as a linear combination of principal components
- How many do we need?



(Eigen) Face Recognition

- Face similarity
 - in the reduced space
 - insensitive to lighting expression, orientation
- Projecting new “faces”
 - everything is a face



$$x_n \quad z_n = W^T x_n$$

new face (not in training)

$$\hat{x}_n = W z_n$$

projected to eigenfaces